

# Lab 4: The Jacobian in Action

EE0849 Introduction to Robotics

Basri Erdogan

## Table of contents

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>1</b> | <b>Lab Overview</b>                                        | <b>2</b>  |
| 1.1      | Objectives . . . . .                                       | 2         |
| 1.2      | Prerequisites . . . . .                                    | 2         |
| 1.3      | Required Materials . . . . .                               | 2         |
| 1.4      | Deliverables . . . . .                                     | 2         |
| <b>2</b> | <b>Part 1: Computing the Jacobian (20 minutes)</b>         | <b>3</b>  |
| 2.1      | 1.1 Setup . . . . .                                        | 3         |
| 2.2      | 1.2 Analytical Jacobian . . . . .                          | 3         |
| 2.3      | 1.3 Exercise 1: Verify the Jacobian . . . . .              | 3         |
| <b>3</b> | <b>Part 2: Jacobian Column Interpretation (15 minutes)</b> | <b>5</b>  |
| 3.1      | 2.1 Visualizing Jacobian Columns . . . . .                 | 5         |
| 3.2      | 2.2 Exercise 2: Velocity Computation . . . . .             | 5         |
| <b>4</b> | <b>Part 3: Singularity Analysis (25 minutes)</b>           | <b>7</b>  |
| 4.1      | 3.1 Singularity Condition . . . . .                        | 7         |
| 4.2      | 3.2 Exercise 3: Explore Singularities . . . . .            | 7         |
| 4.3      | 3.3 Visualize Lost Mobility Direction . . . . .            | 7         |
| <b>5</b> | <b>Part 4: Inverse Velocity Kinematics (20 minutes)</b>    | <b>9</b>  |
| 5.1      | 4.1 Direct Inverse . . . . .                               | 9         |
| 5.2      | 4.2 Exercise 4: Inverse Velocity . . . . .                 | 9         |
| 5.3      | 4.3 Damped Least Squares . . . . .                         | 9         |
| <b>6</b> | <b>Part 5: Resolved-Rate Motion Control (25 minutes)</b>   | <b>11</b> |
| 6.1      | 5.1 Single Step . . . . .                                  | 11        |
| 6.2      | 5.2 Exercise 5: Point-to-Point Motion . . . . .            | 11        |
| 6.3      | 5.3 Exercise 6: Line Following . . . . .                   | 12        |
| <b>7</b> | <b>Part 6: Manipulability Ellipse (20 minutes)</b>         | <b>13</b> |
| 7.1      | 6.1 Computing the Ellipse . . . . .                        | 13        |
| 7.2      | 6.2 Exercise 7: Visualize Manipulability . . . . .         | 13        |
| 7.3      | 6.3 Exercise 8: Manipulability Map . . . . .               | 14        |
| <b>8</b> | <b>Summary and Cleanup</b>                                 | <b>15</b> |
| 8.1      | Checklist . . . . .                                        | 15        |
| 8.2      | Key Takeaways . . . . .                                    | 15        |
| 8.3      | Next Week . . . . .                                        | 15        |

# 1 Lab Overview

## 1.1 Objectives

In this lab, you will:

1. Compute and verify the analytical Jacobian for a 2-link arm
2. Visualize Jacobian columns as velocity contributions
3. Detect and analyze singularities
4. Implement resolved-rate motion control
5. Visualize manipulability ellipses
6. Compare analytical vs numerical Jacobians

## 1.2 Prerequisites

- Completed Lab 3 (inverse kinematics)
- Understanding of partial derivatives
- Linear algebra basics (matrix inverse, determinant, SVD)

## 1.3 Required Materials

- Computer with Python environment
- Lab notebook for calculations

## 1.4 Deliverables

- Working Jacobian computation function
- Singularity detection and visualization
- Resolved-rate trajectory following
- Manipulability map visualization

## 2 Part 1: Computing the Jacobian (20 minutes)

### 2.1 1.1 Setup

Create a new file `lab4_jacobian.py`:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Ellipse
from matplotlib.animation import FuncAnimation

# Link parameters
L1 = 1.0 # meters
L2 = 0.5 # meters

def fk_2link(theta1, theta2, L1, L2):
    """Forward kinematics for 2-link arm."""
    x = L1 * np.cos(theta1) + L2 * np.cos(theta1 + theta2)
    y = L1 * np.sin(theta1) + L2 * np.sin(theta1 + theta2)
    return np.array([x, y])
```

### 2.2 1.2 Analytical Jacobian

Implement the analytical Jacobian:

```
def jacobian_2link(theta1, theta2, L1, L2):
    """
    Compute the 2x2 Jacobian for a 2-link planar arm.

    J = [ -L1*s1 - L2*s12, -L2*s12 ]
         [ L1*c1 + L2*c12, L2*c12 ]
    """
    s1 = np.sin(theta1)
    c1 = np.cos(theta1)
    s12 = np.sin(theta1 + theta2)
    c12 = np.cos(theta1 + theta2)

    J = np.array([
        [-L1*s1 - L2*s12, -L2*s12],
        [L1*c1 + L2*c12, L2*c12]
    ])
    return J
```

### 2.3 1.3 Exercise 1: Verify the Jacobian

Test your Jacobian at a specific configuration:

```
# Test configuration
theta1, theta2 = np.radians(45), np.radians(60)

# Compute Jacobian
J = jacobian_2link(theta1, theta2, L1, L2)
print("Jacobian at 45°, 60°:")
print(J)
print(f"\nDeterminant: {np.linalg.det(J):.6f}")
```

Verify numerically using finite differences:

```
def jacobian_numerical(theta1, theta2, L1, L2, epsilon=1e-6):
    """Compute Jacobian numerically using finite differences."""
    J = np.zeros((2, 2))

    # Partial derivatives with respect to theta1
    x_plus = fk_2link(theta1 + epsilon, theta2, L1, L2)
    x_minus = fk_2link(theta1 - epsilon, theta2, L1, L2)
    J[:, 0] = (x_plus - x_minus) / (2 * epsilon)

    # Partial derivatives with respect to theta2
    x_plus = fk_2link(theta1, theta2 + epsilon, L1, L2)
    x_minus = fk_2link(theta1, theta2 - epsilon, L1, L2)
    J[:, 1] = (x_plus - x_minus) / (2 * epsilon)

    return J

# Compare
J_analytical = jacobian_2link(theta1, theta2, L1, L2)
J_numerical = jacobian_numerical(theta1, theta2, L1, L2)

print("Analytical Jacobian:")
print(J_analytical)
print("\nNumerical Jacobian:")
print(J_numerical)
print(f"\nMax difference: {np.max(np.abs(J_analytical - J_numerical)):.2e}")
```

Record your results:

| Entry    | Analytical | Numerical | Difference |
|----------|------------|-----------|------------|
| $J_{11}$ |            |           |            |
| $J_{12}$ |            |           |            |
| $J_{21}$ |            |           |            |
| $J_{22}$ |            |           |            |

## 3 Part 2: Jacobian Column Interpretation (15 minutes)

### 3.1 2.1 Visualizing Jacobian Columns

Each column of  $J$  shows the end-effector velocity when that joint moves alone:

```
def plot_jacobian_columns(theta1, theta2, L1, L2):
    """Visualize Jacobian columns as velocity vectors."""
    fig, ax = plt.subplots(figsize=(10, 8))

    # Plot arm
    x0, y0 = 0, 0
    x1 = L1 * np.cos(theta1)
    y1 = L1 * np.sin(theta1)
    x2 = x1 + L2 * np.cos(theta1 + theta2)
    y2 = y1 + L2 * np.sin(theta1 + theta2)

    ax.plot([x0, x1, x2], [y0, y1, y2], 'ko-', linewidth=4, markersize=12)

    # Jacobian
    J = jacobian_2link(theta1, theta2, L1, L2)

    # Scale for visualization
    scale = 0.3

    # Column 1: effect of theta1
    ax.arrow(x2, y2, scale*J[0, 0], scale*J[1, 0], head_width=0.05,
            head_length=0.03, fc='blue', ec='blue', linewidth=2)
    ax.text(x2 + scale*J[0, 0], y2 + scale*J[1, 0] + 0.1,
            r'$\mathbf{j}_1$ ($=1$)', color='blue', fontsize=12)

    # Column 2: effect of theta2
    ax.arrow(x2, y2, scale*J[0, 1], scale*J[1, 1], head_width=0.05,
            head_length=0.03, fc='red', ec='red', linewidth=2)
    ax.text(x2 + scale*J[0, 1] + 0.1, y2 + scale*J[1, 1],
            r'$\mathbf{j}_2$ ($=1$)', color='red', fontsize=12)

    ax.set_xlim(-0.5, 2)
    ax.set_ylim(-0.5, 1.5)
    ax.set_aspect('equal')
    ax.grid(True, alpha=0.3)
    ax.set_title(f'Jacobian Columns at {np.degrees(theta1):.0f}°, {np.degrees(theta2):.0f}°')

    plt.tight_layout()
    return fig

fig = plot_jacobian_columns(np.radians(45), np.radians(60), L1, L2)
plt.savefig('jacobian_columns.png', dpi=150)
plt.show()
```

### 3.2 2.2 Exercise 2: Velocity Computation

Given joint velocities, compute end-effector velocity:

```

# Joint velocities
theta1_dot = 0.5 # rad/s
theta2_dot = -0.3 # rad/s

# Configuration
theta1, theta2 = np.radians(45), np.radians(60)

# Compute end-effector velocity
J = jacobian_2link(theta1, theta2, L1, L2)
q_dot = np.array([theta1_dot, theta2_dot])
v = J @ q_dot

print(f"Joint velocities:  $\dot{\theta}_1$  = {theta1_dot} rad/s,  $\dot{\theta}_2$  = {theta2_dot} rad/s")
print(f"End-effector velocity:  $\dot{x}$  = {v[0]:.4f} m/s,  $\dot{y}$  = {v[1]:.4f} m/s")
print(f"Speed: {np.linalg.norm(v):.4f} m/s")

```

**Record:**  $\dot{x}$  = \_\_\_\_\_ m/s,  $\dot{y}$  = \_\_\_\_\_ m/s

## 4 Part 3: Singularity Analysis (25 minutes)

### 4.1 3.1 Singularity Condition

For our 2-link arm, singularities occur when:

$$\det(J) = L_1 L_2 \sin \theta_2 = 0$$

```
def singularity_measure(theta2, L1, L2):
    """Compute det(J) = L1*L2*sin(theta2)."""
    return L1 * L2 * np.sin(theta2)

def is_singular(theta2, threshold=0.01):
    """Check if configuration is singular."""
    return abs(np.sin(theta2)) < threshold
```

### 4.2 3.2 Exercise 3: Explore Singularities

Test configurations near and at singularities:

```
test_configs = [
    (np.radians(45), np.radians(60), "Normal"),
    (np.radians(30), np.radians(5), "Near singular"),
    (np.radians(30), 0, "Singular (extended)"),
    (np.radians(45), np.pi, "Singular (folded)"),
]

print("Singularity Analysis")
print("=" * 60)
for theta1, theta2, description in test_configs:
    J = jacobian_2link(theta1, theta2, L1, L2)
    det_J = np.linalg.det(J)
    x, y = fk_2link(theta1, theta2, L1, L2)

    print(f"\n{description}:")
    print(f"    theta2 = {np.degrees(theta2):.1f}°")
    print(f"    det(J) = {det_J:.6f}")
    print(f"    EE position: ({x:.3f}, {y:.3f})")
    print(f"    Distance from origin: {np.sqrt(x**2 + y**2):.3f} m")
```

#### Questions:

1. At singular configurations, where is the end-effector located?  
*Answer:*
2. What is the relationship between  $\det(J) = 0$  and workspace boundaries?

*Answer:*

### 4.3 3.3 Visualize Lost Mobility Direction

At a singularity, we lose mobility in one direction:

```
def visualize_singularity(theta1, theta2, L1, L2):
    """Visualize what happens at/near singularity."""
    fig, ax = plt.subplots(figsize=(10, 8))
```

```

# Draw workspace boundaries
t = np.linspace(0, 2*np.pi, 100)
r_max = L1 + L2
r_min = abs(L1 - L2)
ax.plot(r_max * np.cos(t), r_max * np.sin(t), 'g--', alpha=0.5)
ax.plot(r_min * np.cos(t), r_min * np.sin(t), 'r--', alpha=0.5)

# Plot arm
x1 = L1 * np.cos(theta1)
y1 = L1 * np.sin(theta1)
x2, y2 = fk_2link(theta1, theta2, L1, L2)

ax.plot([0, x1, x2], [0, y1, y2], 'ko-', linewidth=4, markersize=12)

# Jacobian and SVD
J = jacobian_2link(theta1, theta2, L1, L2)
U, S, Vt = np.linalg.svd(J)

# Draw principal directions
scale = 0.3
for i, (sigma, color) in enumerate(zip(S, ['green', 'red'])):
    direction = U[:, i]
    ax.arrow(x2, y2, scale*sigma*direction[0], scale*sigma*direction[1],
            head_width=0.04, fc=color, ec=color, linewidth=2)

ax.set_title(f'={S[0]:.3f},   ={S[1]:.3f}, det(J)={np.linalg.det(J):.4f}')
ax.set_xlim(-2, 2)
ax.set_ylim(-1, 2)
ax.set_aspect('equal')
ax.grid(True, alpha=0.3)

return fig

# Near singularity
fig = visualize_singularity(np.radians(30), np.radians(5), L1, L2)
plt.savefig('near_singularity.png', dpi=150)
plt.show()

```



## 5 Part 4: Inverse Velocity Kinematics (20 minutes)

### 5.1 4.1 Direct Inverse

When not at a singularity, compute joint velocities directly:

```
def inverse_velocity(v_desired, theta1, theta2, L1, L2):
    """Compute joint velocities for desired end-effector velocity."""
    J = jacobian_2link(theta1, theta2, L1, L2)

    if abs(np.linalg.det(J)) < 1e-6:
        raise ValueError("At or near singularity!")

    q_dot = np.linalg.inv(J) @ v_desired
    return q_dot
```

### 5.2 4.2 Exercise 4: Inverse Velocity

Given desired end-effector velocity, find joint velocities:

```
# Desired velocity
v_desired = np.array([0.1, 0.2]) # m/s

# Configuration (not singular)
theta1, theta2 = np.radians(45), np.radians(60)

# Compute joint velocities
q_dot = inverse_velocity(v_desired, theta1, theta2, L1, L2)
print(f"Desired EE velocity: [{v_desired[0]:.3f}, {v_desired[1]:.3f}] m/s")
print(f"Required joint velocities: ' = {q_dot[0]:.4f}, ' = {q_dot[1]:.4f} rad/s")

# Verify
J = jacobian_2link(theta1, theta2, L1, L2)
v_check = J @ q_dot
print(f"Verification: [{v_check[0]:.6f}, {v_check[1]:.6f}] m/s")
```

### 5.3 4.3 Damped Least Squares

For robustness near singularities:

```
def inverse_velocity_dls(v_desired, theta1, theta2, L1, L2, damping=0.05):
    """Damped least squares inverse."""
    J = jacobian_2link(theta1, theta2, L1, L2)
    JJT = J @ J.T
    q_dot = J.T @ np.linalg.inv(JJT + damping**2 * np.eye(2)) @ v_desired
    return q_dot

# Test near singularity
theta1, theta2 = np.radians(30), np.radians(3) # Near singular!

try:
    q_dot_direct = inverse_velocity(v_desired, theta1, theta2, L1, L2)
    print(f"Direct inverse: {q_dot_direct}")
except ValueError as e:
    print(f"Direct inverse failed: {e}")
```

```
q_dot_dls = inverse_velocity_dls(v_desired, theta1, theta2, L1, L2, damping=0.1)
print(f"DLS inverse: {q_dot_dls}")
```

## 6 Part 5: Resolved-Rate Motion Control (25 minutes)

### 6.1 5.1 Single Step

```
def resolved_rate_step(q, x_desired, L1, L2, K=1.0, dt=0.01, damping=0.05):
    """One step of resolved-rate control."""
    # Current position
    x_current = fk_2link(q[0], q[1], L1, L2)

    # Error
    error = np.array(x_desired) - x_current

    # Desired velocity
    v_desired = K * error

    # Joint velocities (DLS for robustness)
    q_dot = inverse_velocity_dls(v_desired, q[0], q[1], L1, L2, damping)

    # Integrate
    q_new = q + q_dot * dt

    return q_new, np.linalg.norm(error)
```

### 6.2 5.2 Exercise 5: Point-to-Point Motion

Move the arm to a target using resolved-rate control:

```
def move_to_point(q_start, x_target, L1, L2, max_steps=1000, tol=0.001):
    """Move to target point using resolved-rate control."""
    q = np.array(q_start)
    trajectory = [q.copy()]
    errors = []

    for step in range(max_steps):
        q, error = resolved_rate_step(q, x_target, L1, L2)
        trajectory.append(q.copy())
        errors.append(error)

        if error < tol:
            print(f"Converged in {step+1} steps")
            break

    return np.array(trajectory), errors

# Start and target
q_start = [np.radians(30), np.radians(45)]
x_target = [0.8, 0.6]

trajectory, errors = move_to_point(q_start, x_target, L1, L2)

# Plot convergence
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(errors)
```

```

plt.xlabel('Step')
plt.ylabel('Position Error (m)')
plt.title('Convergence')
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(np.degrees(trajjectory[:, 0]), label=' ')
plt.plot(np.degrees(trajjectory[:, 1]), label=' ')
plt.xlabel('Step')
plt.ylabel('Joint Angle (°)')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.savefig('resolved_rate_convergence.png', dpi=150)
plt.show()

```

### 6.3 5.3 Exercise 6: Line Following

Follow a straight line path:

```

def follow_line(q_start, p_start, p_end, L1, L2, n_waypoints=20):
    """Follow a straight line from p_start to p_end."""
    # Generate waypoints
    waypoints = np.linspace(p_start, p_end, n_waypoints)

    q = np.array(q_start)
    full_trajectory = []

    for waypoint in waypoints:
        # Move to each waypoint
        for _ in range(50): # Max iterations per waypoint
            q, error = resolved_rate_step(q, waypoint, L1, L2, K=2.0)
            full_trajectory.append(q.copy())
            if error < 0.005:
                break

    return np.array(full_trajectory)

# Follow a horizontal line
q_start = [np.radians(60), np.radians(30)]
p_start = [0.6, 0.8]
p_end = [1.2, 0.8]

trajectory = follow_line(q_start, p_start, p_end, L1, L2)
print(f"Total steps: {len(trajectory)}")

```

## 7 Part 6: Manipulability Ellipse (20 minutes)

### 7.1 6.1 Computing the Ellipse

```
def compute_manipulability_ellipse(theta1, theta2, L1, L2):
    """Compute manipulability ellipse parameters."""
    J = jacobian_2link(theta1, theta2, L1, L2)
    U, S, Vt = np.linalg.svd(J)

    # Ellipse parameters
    angle = np.arctan2(U[1, 0], U[0, 0])
    width = S[0] # Major axis (scaled)
    height = S[1] # Minor axis

    # Manipulability measure
    w = S[0] * S[1]

    # Condition number
    kappa = S[0] / S[1] if S[1] > 1e-10 else np.inf

    return angle, width, height, w, kappa
```

### 7.2 6.2 Exercise 7: Visualize Manipulability

```
def plot_with_ellipse(theta1, theta2, L1, L2, scale=0.3):
    """Plot arm with manipulability ellipse."""
    fig, ax = plt.subplots(figsize=(10, 8))

    # Arm
    x1 = L1 * np.cos(theta1)
    y1 = L1 * np.sin(theta1)
    x2, y2 = fk_2link(theta1, theta2, L1, L2)
    ax.plot([0, x1, x2], [0, y1, y2], 'ko-', linewidth=4, markersize=12)

    # Ellipse
    angle, width, height, w, kappa = compute_manipulability_ellipse(
        theta1, theta2, L1, L2)

    ellipse = Ellipse((x2, y2), scale*width, scale*height,
                      angle=np.degrees(angle), fill=True, alpha=0.3,
                      color='purple')
    ax.add_patch(ellipse)

    # Info
    ax.set_title(f'w = {w:.4f}, kappa = {kappa:.2f}')
    ax.set_xlim(-0.5, 2)
    ax.set_ylim(-0.5, 1.5)
    ax.set_aspect('equal')
    ax.grid(True)

    return fig

# Test at different configurations
```

```

configs = [
    (np.radians(45), np.radians(90)), # Elbow bent
    (np.radians(30), np.radians(30)), # Slightly bent
    (np.radians(20), np.radians(5)),  # Nearly extended
]

for theta1, theta2 in configs:
    fig = plot_with_ellipse(theta1, theta2, L1, L2)
    plt.savefig(f'ellipse_{int(np.degrees(theta2))}.png', dpi=150)
    plt.show()

```

## 7.3 6.3 Exercise 8: Manipulability Map

Create a map showing manipulability over configuration space:

```

def manipulability_map(L1, L2, resolution=50):
    """Create manipulability map."""
    theta1_range = np.linspace(-np.pi, np.pi, resolution)
    theta2_range = np.linspace(-np.pi, np.pi, resolution)
    T1, T2 = np.meshgrid(theta1_range, theta2_range)

    W = np.zeros_like(T1)
    for i in range(resolution):
        for j in range(resolution):
            J = jacobian_2link(T1[i, j], T2[i, j], L1, L2)
            JJT = J @ J.T
            det = np.linalg.det(JJT)
            W[i, j] = np.sqrt(det) if det > 0 else 0

    return T1, T2, W

T1, T2, W = manipulability_map(L1, L2)

plt.figure(figsize=(10, 8))
plt.contourf(np.degrees(T1), np.degrees(T2), W, levels=30, cmap='viridis')
plt.colorbar(label='Manipulability w')
plt.contour(np.degrees(T1), np.degrees(T2), W, levels=[0.01], colors='red')
plt.xlabel(' (degrees)')
plt.ylabel(' (degrees)')
plt.title('Manipulability Map (Red = Singularity)')
plt.savefig('manipulability_map.png', dpi=150)
plt.show()

```

## 8 Summary and Cleanup

### 8.1 Checklist

- ☐ `jacobian_2link()` implemented and verified against numerical
- ☐ Jacobian column visualization complete
- ☐ Singularity analysis ( $\det(J) = 0$  at  $\theta = 0, \pm\pi$ )
- ☐ `inverse_velocity_dls()` working near singularities
- ☐ Resolved-rate control converging to target
- ☐ Line following demonstrated
- ☐ Manipulability ellipse visualization
- ☐ Manipulability map generated

### 8.2 Key Takeaways

1. The Jacobian maps joint velocities to end-effector velocities
2. Each column shows one joint's contribution to EE motion
3. Singularities ( $\det(J) = 0$ ) occur at workspace boundaries
4. Use damped least squares near singularities
5. Manipulability quantifies motion capability

### 8.3 Next Week

Week 6: Robot Dynamics - Equations of Motion

- Newton-Euler formulation
- Lagrangian dynamics
- Mass matrix and inertia
- Gravity compensation